



Harnesses and Multi-Agent Orchestration

From token prediction to coordinated, rollback-safe agent teams

Sangwhan Moon

June 22, 2026

SECTION

Harnesses 101

LLMs become useful agents only after a control loop gives them tools, state, policy, and consequences.

What Is an LLM?

NEXT TOKEN

- A model predicts the next token from the tokens already seen.
- It appends that token and predicts again.
- Long answers are many small probability decisions chained together.

AUTOREGRESSIVE LOOP

The cat sat on the mat

distribution over the next token

mat: 0.42 rug: 0.21 floor: 0.14

The Transformer Makes It Work

SELF-ATTENTION

- Each prediction attends across the available context.
- In causal LLMs, earlier tokens shape the next-token distribution.
- Repeated attention and feed-forward blocks build useful representations.

THE COST

- Attention cost grows with pairs of tokens.
- Bigger context windows still have real compute and reliability costs.
- The attention matrix still scales as $n \times n$.

References: [1]

Tokens, Context, and Sampling

TOKENS

- Models read tokens, not characters.
- Code, prose, logs, and tool output all spend the same budget.

CONTEXT WINDOW

- Prompt, history, tool output, and response share the window.
- More room is not the same as better control.

SAMPLING

- Temperature changes how broadly the model samples.
- Top-k and top-p trim candidate tokens before sampling.

low temperature: narrow

high temperature: exploratory

same prompt, different useful attempts

References: [1]

How Models Are Built

PRE-TRAINING

Broad next-token competence from large-scale data.

MID-TRAINING

Sharper behavior for domains, code, math, and long-context use.

POST-TRAINING

SFT and preference optimization teach instruction-following behavior.

DEPLOYMENT

Product systems wrap the model with policy, tools, and constraints.

Base Model to Agent

WHAT GETS ADDED

- SFT teaches instruction following.
- Tool schemas teach requests for external actions.
- The system prompt gives role, scope, and rules.

WHY IT CHANGES BEHAVIOR

- The harness runs the loop.
- It decides what actually happens.
- A tool request becomes a controlled side effect.

base model -> SFT -> tool use -> system prompt -> harness

What the Harness Owns

CONTROL LOOP

- Manages message history.
- Sends tool schemas to the model.
- Executes tool calls and appends results.

SECURITY BOUNDARY

- Stops, retries, asks for approval, or blocks unsafe actions.
- The model can request an action.
- The harness decides whether it runs.

messages -> LLM -> tool call -> executor -> result

SFT Teaches Tool Use

TRAINING SAMPLE

```
user: "Find failing tests."  
assistant: tool_call(run_tests, {})  
tool: "2 failures in parser_test"  
assistant: "The parser edge case fails..."
```

PATTERN LEARNED

- When to call a tool.
- How to structure arguments.
- How to chain calls.
- How to ground the final response in results.

Training transcripts must stay precise because they become the evidence trail the model sees.

SECTION

Limits of Single Agents

One long session is easy to start, but brittle when planning, execution, and review all compete for the same context.

Single-Agent Harnesses Hit a Ceiling

THE BOTTLENECK

- One context window carries the whole task.
- One trajectory commits to one set of assumptions.
- One role sees the task through one lens.
- Long sessions make early mistakes expensive.

one agent

->

plan + files + logs + decisions +
patches

References: [7], [13]

One Context Window Is Not a Plan

CAPACITY IS NOT CONTROL

- More tokens let the model see more material.
- They do not force the model to use the right material.
- Retrieval, reasoning, and instruction tracking still compete.

FAILURE SHAPE

- Important facts get buried far apart.
- The session becomes harder to inspect as it grows.
- Debugging means reading the whole transcript.

References: [7]

Long Inputs Degrade Attention in Practice

NEEDLE TESTS

- Synthetic needles expose position-sensitive failures.
- Longer context helps, but reliability still has cliffs.

REAL TASKS

- LongBench covers QA, summarization, few-shot, synthetic, and code tasks.
- Context compression helps weaker models, but does not remove the core risk.

References: [7]

Long Agent Chains Accumulate Errors

CHAIN RISK

- Tool calls introduce new state after every step.
- Bad assumptions can become permanent context.
- One early mistake can steer the whole trajectory.

MULTI-TURN RISK

- Multi-turn conversations can be less reliable than single-turn instructions.
- The failure is often unreliability, not raw lack of ability.

step 1 -> step 2 + small error -> step 3 + larger error -> wrong branch

References: [5], [13]

Decomposition Is the Escape Hatch

WHAT WORKS

- Break the task into smaller subproblems.
- Solve each subproblem with the right context and role.
- Feed results forward only when useful.

WHY MODELS ARE GOOD AT IT

- Frontier models are strong at task structure and intermediate reasoning.
- Least-to-most prompting and Tree of Thoughts point at the same shape.

large task -> A B C -> synthesis

References: [2], [4], [6], [8], [9]

SECTION

Multi-Agent Ensembles

Agent teams adapt ensemble ideas to engineering work: diversity, role fit, critique, and structured synthesis.

Multi-Agents and Ensemble Inspiration

CORE CLAIM

- Agent teams are test-time ensembles with tools, roles, and memory boundaries.
- Diversity matters when the task has many plausible paths.
- Synthesis matters because raw voting is not enough for engineering work.

fan out -> independent work -> critique -> synthesis

References: [3], [10], [11], [12]

Classical Ensemble Ideas Map Cleanly

BAGGING

Repeated samples, parallel agents, and vote-style aggregation.

BOOSTING

Staged correction, review passes, and focused follow-up work.

MIXTURE OF EXPERTS

Role specialization, routing, and fusion across complementary outputs.

References: [3], [10], [11], [12]

Voting Beats One Guess When Errors Differ

SAMPLE DIVERSITY

- Self-consistency samples multiple reasoning paths.
- It selects the answer that is most consistent across paths.
- Gains depend on task difficulty and error diversity.

AGENT FOREST

- More Agents Is All You Need scales the same intuition to instantiated agents.
- Parallel attempts are cheap when orchestration is automated.

path A path B path C -> consistent answer

References: [3], [10]

Specialization Beats One Generalist

ROLE FIT

- Different models and roles are best on different examples.
- Preserve diversity until the final merge.
- Assign roles by failure mode, not personality.

FUSION

- LLM-Blender ranks candidate outputs, then fuses strong pieces.
- Mixture-of-Agents layers multiple model outputs into later synthesis stages.

architect qa security performance -> synthesizer

References: [11], [12]

Debate Adds Friction

DELIBERATION

- Chain-of-thought makes intermediate reasoning available to the model.
- Tree of Thoughts searches multiple candidate paths before committing.

GROUNDING

- ReAct interleaves reasoning and action so the agent can gather evidence.
- Collaboration externalizes deliberation across workers.

References: [2], [5], [6]

Small Contexts Can Beat One Long Context

WHY IT WORKS

- Each agent gets a smaller prompt with fewer distractions.
- Each role can carry a narrower checklist.
- Parallel agents expose disagreements early.

WHERE IT FAILS

- Synthesis must be structured.
- Otherwise the team just creates more text.
- The orchestration layer is where reliability is won or lost.

References: [7], [10], [11], [12]

SECTION

finb Design Decisions

The system assumes agents will fail, so it makes damage cheap, work inspectable, and synthesis explicit.

Design Around Failure

LOCKDOWN

Safe, but the human becomes the bottleneck.

YOLO

Fast, but one bad command can destroy the workspace.

REVERSIBLE

Agents get freedom inside a workspace that can roll back instantly.

Target: controlled blast radius, not constant approval.

Safe-to-Destruct Workspaces

ZFS SNAPSHOTS

- Each agent runs in a copy-on-write snapshot.
- Rollback is near-instant.
- Agents can move fast because destruction is reversible.

CHROOT

- Chroot keeps the agent inside its assigned filesystem.
- No containers or VMs are required for the core isolation model.
- Commit or rollback stays explicit.

workspace -> snapshot clone -> chroot -> agent run -> commit / rollback

Agent Teams Are a DAG

PHASES

- The planner emits a phased execution graph.
- Later phases receive synthesized context from earlier phases.
- Dependencies stay explicit.

PARALLELISM

- Tasks inside a phase run concurrently.
- Fresh agents get bounded context.
- Successful work can be reused without rerunning everything.

plan -> phase 1 agents -> synthesis -> phase 2 agents -> final

Mailbox Turns Agents Into a Team

MESSAGES

- Agents send mail to named teammates.
- Coordination stays auditable.
- No shared hidden state is required.

DELIVERY

- Messages are delivered between phases.
- Recipients see the mail in their next prompt.
- The transcript shows who influenced whom.

lead ml data qa -> mailbox -> next phase

Synthesis Has Influence Tiers

PRIMARY

Drives the narrative and owns final coherence.

SUPPORTING

Shapes major sections, constraints, and tradeoffs.

CONTRIBUTING

Adds evidence, edge cases, and checks.

tiered outputs

->

attributed or unified
synthesis

->

one deliverable

References: [11], [12]

Iteration and Harness Abstraction

FOLLOW-UP LOOP

- Feed the previous deliverable back into planning.
- Fresh agents run with fresh context.
- Failed agents can be retried without rerunning successful work.

FOUR LAYERS

- TUI: interactive control and session visibility.
- Orchestrator: planning, mailbox, context, synthesis.
- Agent manager: lifecycle and output capture.
- Harness: Claude Code, Gemini CLI, and future tools.

SECTION

Example Results

finb is aimed at concrete multi-file engineering work, not just longer chat completions.

Example Projects

TERMNES

Terminal NES emulator in Rust: CPU, PPU, mappers, renderer, input, and targeted APU support.

AT3RS

ATRAC3 audio codec in Rust from a specialized technical domain.

SLOPPNG

PNG tooling in Rust with binary parsing, validation, and CLI workflows.

These are multi-file systems with tests, integration points, and domain-specific constraints.

What the Results Demonstrate

DECOMPOSITION

Planning agents split hard builds into executable phases.

SPECIALIZATION

Specialists own emulation, binary formats, rendering, QA, and integration.

VERIFICATION

Review passes catch integration bugs single-pass generation leaves behind.

parallel work

->

synthesis

->

coherent codebase

->

verification

References

REFERENCES

- [1] Ashish Vaswani et al., Attention Is All You Need. NeurIPS 2017.
- [2] Jason Wei et al., Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022.
- [3] Xuezhi Wang et al., Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR 2023.
- [4] Denny Zhou et al., Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. ICLR 2023.
- [5] Shunyu Yao et al., ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.
- [6] Shunyu Yao et al., Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS 2023.

References

REFERENCES

- [7] Yushi Bai et al., LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. ACL 2024.
- [8] Xiao Liu et al., AgentBench: Evaluating LLMs as Agents. ICLR 2024.
- [9] Shuyan Zhou et al., WebArena: A Realistic Web Environment for Building Autonomous Agents. ICLR 2024.
- [10] Junyou Li et al., More Agents Is All You Need. TMLR 2024.
- [11] Dongfu Jiang et al., LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion. ACL 2023.
- [12] Junlin Wang et al., Mixture-of-Agents Enhances Large Language Model Capabilities. ICLR 2025.
- [13] Philippe Laban et al., LLMs Get Lost In Multi-Turn Conversation. ICLR 2026.

Questions?

Harnesses & Multi-Agent Orchestration